

集成学习

Ensemble Learning

明玉瑞 Yurui Ming
yrming@gmail.com

声明

Disclaimer

- 本讲义在准备过程中由于时间所限，所用材料来源并未规范标示引用来源。所引材料仅用于教学所用，作者无意侵犯原著者之知识产权，所引材料之知识产权均归原著者所有；若原著者介意之，请联系作者更正及删除。

These slides have been developed free-of-charge for educational purposes only and without any commercial interest. However, time limit during the preparation incurs the situation that not all the sources of the materials (texts or images) are properly referenced or clearly manifested. But all contents are solely for teaching and the author is with no intention to infringe the copyright bestowed on the originators and all credits go to corresponding IP holders. Please address the author have you had any concern so he can act on a remedy including deletion per request.

思想与定义

Idea & Definition

- 集成学习是通过将多个模型进行组合，得到比任何单一模型都更优的预测效果的方法，其核心思想是所谓的“群体智慧”。一般地，通过合适地组合产生不同误差的单个模型，可以降低整体误差。

Ensemble Learning is to combine multiple models to make better predictions than any single model could achieve alone. The core idea is the so-called the wisdom of crowds. Generally, individual models make different mistakes, by combining them intelligently, we can reduce overall error.

- 简单的集成学习方法在分类问题里为“多数表决”，假设对于二分类问题，有三个模型，有两个模型判断是阳性，一个模型判断是阴性，结果便定为阳性。在回归问题是“求平均”，即将几个模型的预测值平均之后，作为最终的预测值。

Simple ensemble learning methods use majority voting for classification problems. For a binary classification task, suppose there are three models: two predict positive and one predicts negative. The final result will be positive. For regression problems, the method is averaging: the final prediction is the average of the predictions from multiple models.

有效性

Effectiveness

► 集成学习有效的原因可以从三个方面分析：

- 从统计层面来讲，数据有限会导致存在多个合理的假设。这些假设的泛化能力不尽相同，组合模型，可以降低挑选到最差的泛化能力模型的概率；
- 从计算层面来讲，模型在训练过程中容易陷入局部最优，通过组合模型，能从局部最优到全局次优，提高模型的泛化性能；
- 从表达层面来讲，单个模型本身存在表达能力局限。而在机器学习中，我们需要平衡偏差与方差，组合模型有可能在这方面做的更好。

The reasons why ensemble learning works can be analyzed from three aspects:

- From a **statistical perspective**, limited data leads to multiple reasonable hypotheses with varying generalization abilities. Combining models reduces the probability of selecting a model with poor generalization.
- From a **computational perspective**, models tend to get stuck in local optima during training. By combining multiple models, we can move from local optima to a globally near-optimal solution, improving generalization performance.
- From a **representational perspective**, individual models have inherent limitations in expressive power. In machine learning, we need to balance bias and variance, and model combinations can often achieve a better trade-off in this regard.

有效性

Effectiveness

► 基于上面三个方面，集成学习要能够有效地工作，基（础）模型得满足下面两个条件：

- 准确性：效果优于随机猜测，即错误率 $<50\%$ （优于抛硬币）
- 多样性：在新数据上产生不同的错误，即模型在不同样本上的预测结果不一致

Based on the above, for an ensemble to outperform individual members, models must be:

- Accurate: Better than random guessing, aka., error rate $< 50\%$ (better than coin flip).
 - Diverse: Make different errors on new data, aka., models disagree on different examples.
- 例如，三个基模型的分类的错误率为 $\varepsilon = 0.3$ ，则根据多数表决准则，集成模型的犯错是两个模型或三个模型都犯错，其错误率为 $C_3^2 \varepsilon^2 (1 - \varepsilon) + C_3^3 \varepsilon^3 = 0.216$ ，其小于基模型的0.3。

For example, if the classification error rate of each of the three base models is $\varepsilon = 0.3$, then under the majority voting rule, the ensemble model makes a mistake only when two or all three models are wrong. Its error rate is $C_3^2 \varepsilon^2 (1 - \varepsilon) + C_3^3 \varepsilon^3 = 0.216 < 0.3$, which is less than that of base model with $\varepsilon = 0.3$.

有效性

Effectiveness

► 基于上面三个方面，集成学习要能够有效地工作，基（础）模型得满足下面两个条件：

- 准确性：效果优于随机猜测，即错误率 $<50\%$ （优于抛硬币）
- 多样性：在新数据上产生不同的错误，即模型在不同样本上的预测结果不一致

Based on the above, for an ensemble to outperform individual members, models must be:

- Accurate: Better than random guessing, aka., error rate $< 50\%$ (better than coin flip).
 - Diverse: Make different errors on new data, aka., models disagree on different examples.
- 例如，三个基模型的分类的错误率为 $\varepsilon = 0.3$ ，则根据多数表决准则，集成模型的犯错是两个模型或三个模型都犯错，其错误率为 $C_3^2 \varepsilon^2 (1 - \varepsilon) + C_3^3 \varepsilon^3 = 0.216$ ，其小于基模型的0.3。

For example, if the classification error rate of each of the three base models is $\varepsilon = 0.3$, then under the majority voting rule, the ensemble model makes a mistake only when two or all three models are wrong. Its error rate is $C_3^2 \varepsilon^2 (1 - \varepsilon) + C_3^3 \varepsilon^3 = 0.216 < 0.3$, which is less than that of base model with $\varepsilon = 0.3$.

有效性

Effectiveness

- ▶ 基于集成学习有效的情形，我们可以推出集成学习不工作的情况：
 - ▶ 基础模型质量差：如果单个模型准确率 $<50\%$ ，即模型效果比随机猜测还差，则集成模型并不能工作，即所谓的垃圾数据进来 → 垃圾结果出去
 - ▶ 缺乏多样性：如果所有基模型都犯相同的错误，或模型预测之间相关性很高，则由于模型之间没有互补优势，因此集成模型也不能很好地工作

From the conditions under which ensemble learning works, we can derive when it fails:

- ▶ Poor base models: If the accuracy of individual models is below 50%, meaning they perform worse than random guessing, the ensemble will not work. This is the classic case of garbage in, garbage out.
- ▶ Lack of diversity: If all base models make the same mistakes, or their predictions are highly correlated, the models have no complementary strengths, so the ensemble will not perform well either.

Bagging

- 我们介绍的第一种集成学习算法称为bagging算法，关键思想即便使用相同的算法、相同原始数据，只要在不同子集上训练，就能得到不同的模型。那么，如何从一个数据集中得到多组不同的训练集？解决方案是采用Bootstrap采样，即有放回抽样。因此，Bagging等于自助采样加聚合，目标是从单一数据集中构建多样化模型，以降低方差。

The first ensemble learning method to be introduced is named bagging. The key insight of it is, even with the same algorithm and same data, we can still create different models by training on different subsets. Now the problem is how do we get different training sets from one dataset? The solution is bootstrap sampling, or sampling with replacement. Therefore, bagging equals bootstrap plus aggregation. To sum it up, the goal of bagging is to create diverse models from a single dataset to reduce variance.

- 考虑数据集 $\mathcal{D} = \{s_1, s_2, \dots, s_n\}$ ，对每个模型，通过自助采样，或有放回地抽取 n 个样本来构建新数据集。例如，对自助采样集1， $\mathcal{D}_1 = \{s_1, s_3, s_6, s_1, s_5, \dots\}$ ，注意 s_1 出现了两次。对自助采样集2， $\mathcal{D}_2 = \{s_2, s_4, s_1, s_n, s_3, \dots\}$ 。因为每个自助采样集都略有不同，从而得到多样化的模型。

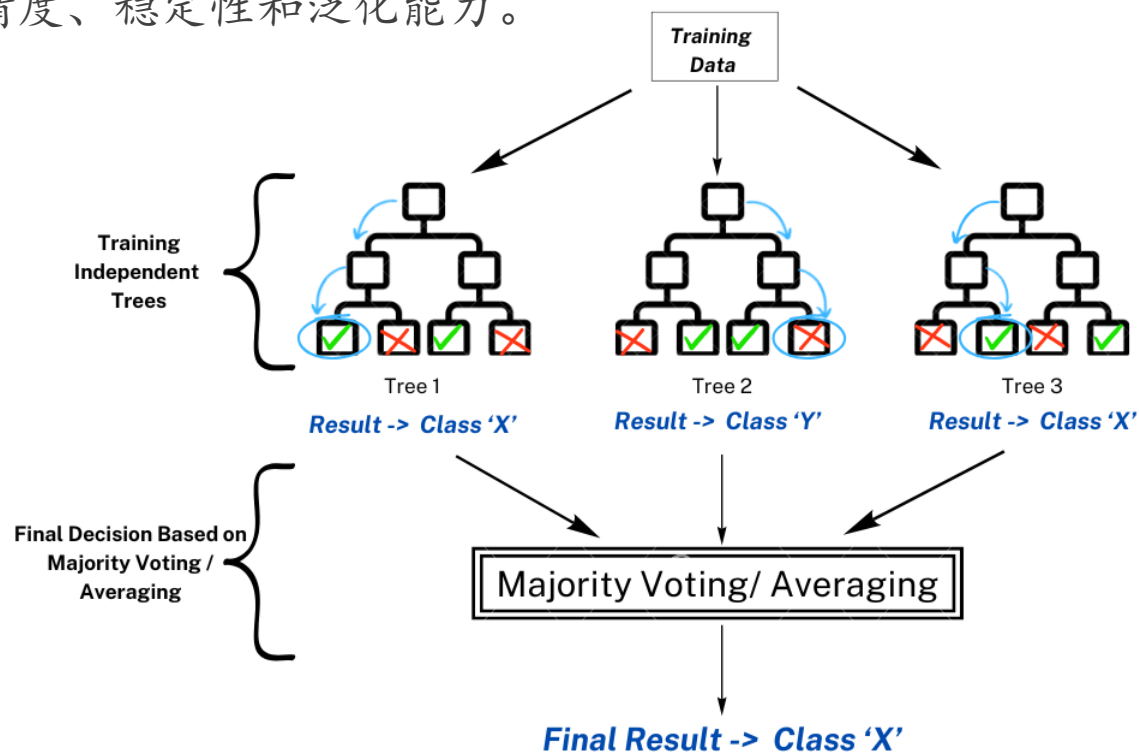
Consider the dataset $\mathcal{D} = \{s_1, s_2, \dots, s_n\}$. For each model, create new dataset by sampling n examples with replacement. For example, for bootstrap sample 1, $\mathcal{D}_1 = \{s_1, s_3, s_6, s_1, s_5, \dots\}$, Notice that s_1 appears twice. For bootstrap sample 2, $\mathcal{D}_2 = \{s_2, s_4, s_1, s_n, s_3, \dots\}$. Since each bootstrap sample is slightly different, we obtain diverse models.

随机森林

Random Forest

- 一般数据集的样本都有多个属性组成，除了可以对样本进行采样，也可以对属性进行自助采样。结合自助采样和特征随机选择两种技术，基于集成学习思想，基模型由决策树组成的机器学习算法，称为随机森林。其由学者Leo Breiman于2001年提出，通过集成学习整合多棵决策树的预测结果，从而提升模型的预测精度、稳定性和泛化能力。

Generally, samples of a dataset can have multiple attributes or features. Besides sampling individual samples from the dataset, we can also sample attributes. The method that combines two core technologies, aka., bootstrap sampling and random feature selection, based on the idea of Ensemble Learning, with base models decision trees, is called random forest. It is proposed by Leo Breiman in 2001, by integrating the prediction results of multiple decision trees to improve the prediction accuracy, stability, and generalization ability of the model.



随机森林

Random Forest

- ▶ 随机森林的核心优势在于，它有效解决了单棵决策树容易过拟合、泛化能力弱的问题。由于每棵决策树都基于不同的自助采样集训练，且在每个决策节点分裂时仅随机选择部分特征，使得单棵树的预测误差能够相互抵消，最终输出更稳健、更可靠的结果。同时，随机森林具有良好的抗噪声能力，对缺失值、异常值不敏感，无需复杂的特征预处理，训练效率高且易于实现，广泛应用于分类、回归、特征重要性评估、异常检测等多个机器学习领域，例如金融风险预测、医疗诊断、图像识别、环境监测等。

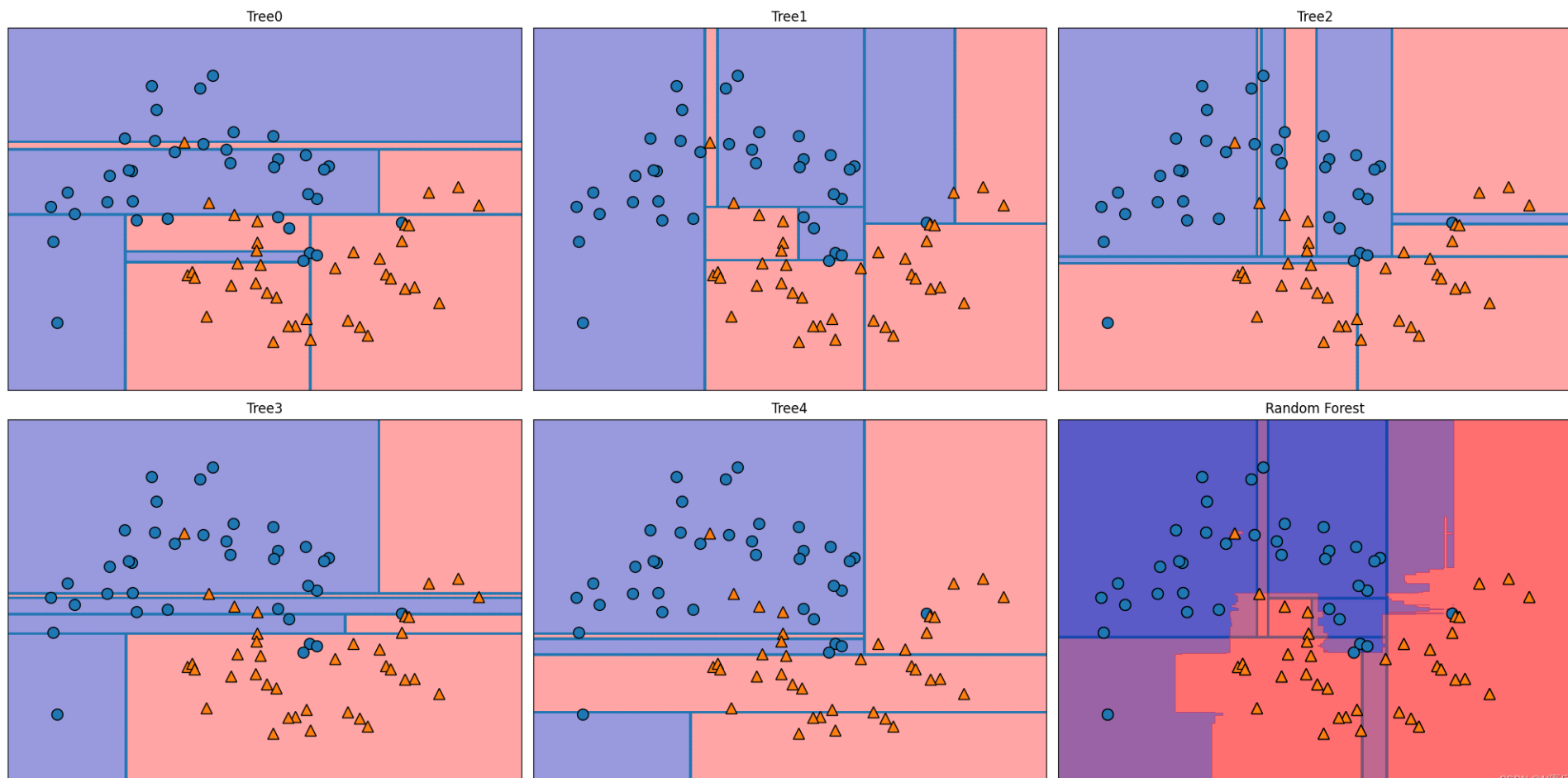
The core advantage of Random Forest is that it effectively solves the problems of overfitting and weak generalization ability of a single decision tree. Since each decision tree is trained on a different bootstrap sample, and only a subset of features is randomly selected when splitting at each decision node, the prediction errors of individual trees can offset each other, and finally output a more robust and reliable result. At the same time, Random Forest has good noise resistance, is insensitive to missing values and outliers, does not require complex feature preprocessing, has high training efficiency and is easy to implement. It is widely used in many machine learning fields such as classification, regression, feature importance evaluation, and anomaly detection, such as financial risk prediction, medical diagnosis, image recognition, environmental monitoring, etc.

随机森林

Random Forest

- 下图展示了随机森林与单个决策树相比时的决策边界，可见其在偏差与方差之间取得了更好的平衡。

The below figure demonstrates the decision boundary between single decision trees and random forest, which is better balanced between bias and variance.



Boosting

- ▶ Bagging中的基模型是并行组合在一起的，如果允许顺序组合，便构成了另一种集成学习方法，即Boosting。其通过将弱学习器按顺序组合，构建一个强大的集成模型。核心想法是，如果一开始没有成功，就针对自己做错的地方再努力尝试。

The base models in Bagging is arranged in a parallel way. If sequential combination is allowed, we have another ensemble learning approach called boosting. It combines weak learners sequentially to create a strong ensemble. The Boosting intuition is, if at first you don't succeed, try harder on what you got wrong.

- ▶ Boosting方法与Bagging方法的区别有：
 - ▶ 顺序性：模型依次构建（而非并行构建）；
 - ▶ 聚焦错误：每个模型都从之前模型的错误中学习；
 - ▶ 降低偏差：将弱学习器转化为强大的集成模型
- ▶ Key Differences of boosting from Bagging:
 - ▶ Sequential: Models built one after another (not in parallel)
 - ▶ Focus on Mistakes: Each model learns from previous model's errors
 - ▶ Reduce Bias: Turn weak learners into strong ensemble

AdaBoost

- ▶ 下面讲boosting算法中典型算法AdaBoost。其核心思想是，每个模型都会自适应地针对前一个模型的错误进行学习。在训练过程中，首先在带权重的数据集上训练弱学习器，然后提高被错误分类样本的权重，通过重复迭代，不断聚焦难以学习的样本，以达到最后的最优分类器。

Now we introduce the representative of boosting: AdaBoost. The core idea is each model adapts to previous model's mistakes. During the training process, first train weak learner on weighted data, then increase weights of misclassified examples. By repetition, focus on "hard" examples for the final optimal classifier.

- ▶ 我们先引入一些记号：

- ▶ $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ 表示有 N 个样本的集合；
- ▶ w_i 表示样本权重，即第 i 个样本在训练中的重要程度；
- ▶ 共有 M 个弱学习器，为 $h_1, h_2, \dots, h_M$ ；
- ▶ α_m 表示学习器权重，即第 m 个弱学习器在最终集成模型中的重要程度。

- ▶ We first introduce some notations: Definition: AdaBoost Notation

- ▶ N training samples: $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$
- ▶ Sample weights: w_i (importance of sample i for training)
- ▶ M weak learners: $h_1, h_2, \dots, h_M$;
- ▶ Learner weights: α_m (importance of learner m in final ensemble)

AdaBoost

- 假设我们的目标是学习一个可以分类右图所示样本分布的数据集的强分类器，即：

Suppose our goal is to learn a strong classifier which can classify the samples as scattered in the right figure:

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$$

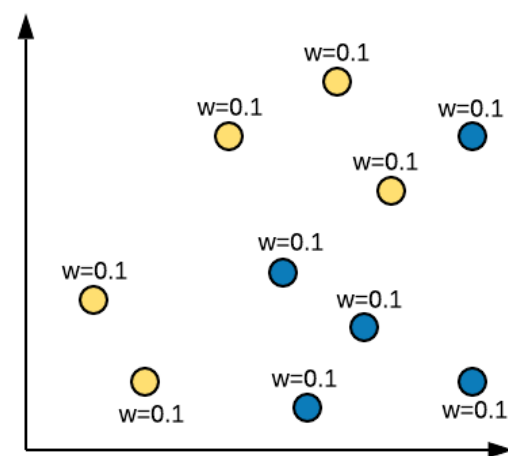
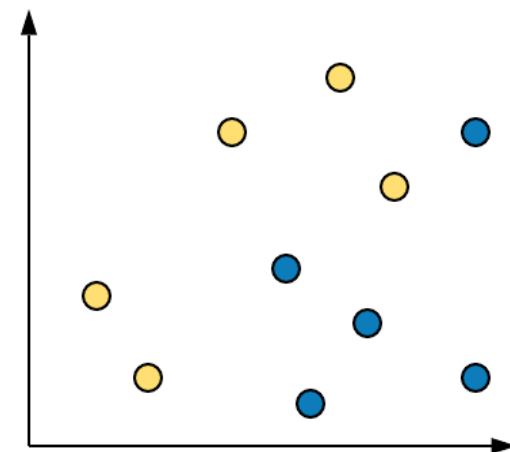
- 步骤1：初始化样本权重。由于一开始没有关于“难样本”的先验知识，因此对所有样本赋相同的初始权重：

Step 1: Initialize sample weights. Since no prior knowledge about “hard” samples, we take the equal importance for all samples:

$$w_i^{(1)} = \frac{1}{N}, i = 1, 2, \dots, N$$

- 权重会在随后的从错误中学习的过程中自适应调整。

Weights adapt as we learn from mistakes in the following.



AdaBoost

- 步骤2: 利用当前的权重 $w_i^{(1)}$ 训练弱分类器 h_1 。注意, h_1 可能会产生一些错误, 但这些错误会指导下一步的学习过程:

Step 2: Train weak learner h_1 using current sample weights $w_i^{(1)}$. Notably, h_1 can makes some mistakes, but these mistakes guide the next learning step.

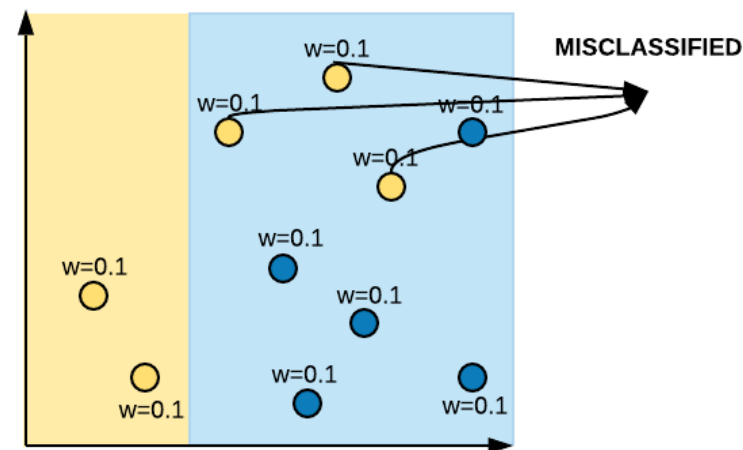
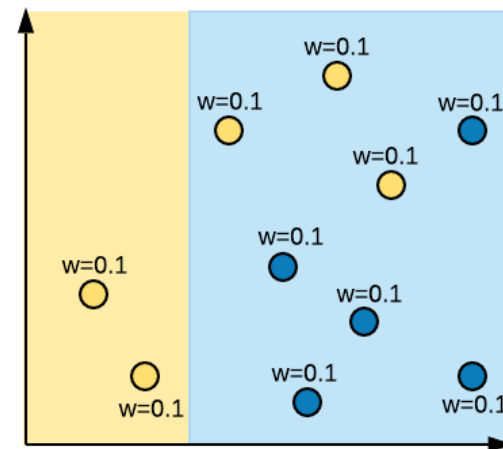
- 步骤3: 按如下公式计算错误权重:

Step 3: Calculate the weighted error according to the following formula:

$$\text{err}_m = \frac{\text{weights of mistakes}}{\text{total weights}} = \frac{\sum_{j \in \{k: h_m(x_k) \neq y_k\}} w_j^{(m)}}{\sum_{i=1}^N w_i^{(m)}}$$

因此, $\text{err}_1 = 0.3$ 。

So we have $\text{err}_1 = 0.3$.



AdaBoost

- 步骤4: 按照以下公式计算分类器权重, 即确定在最终集成模型中对该分类器的置信程度:

Step 4: Calculate classifier weight according to the following formula, or to determine how much to trust this classifier in final ensemble:

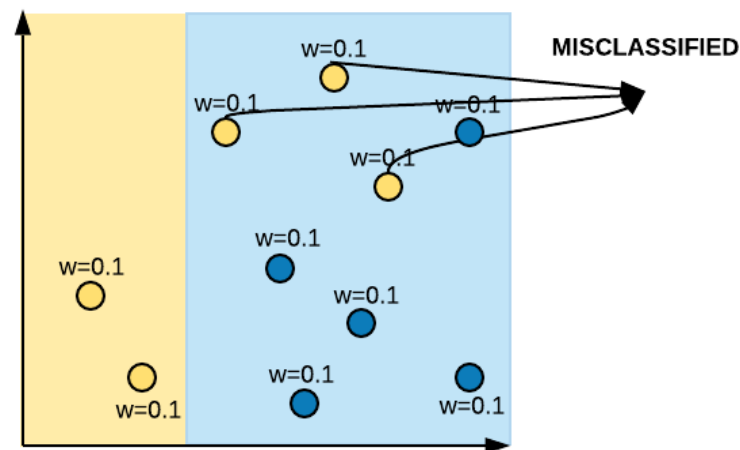
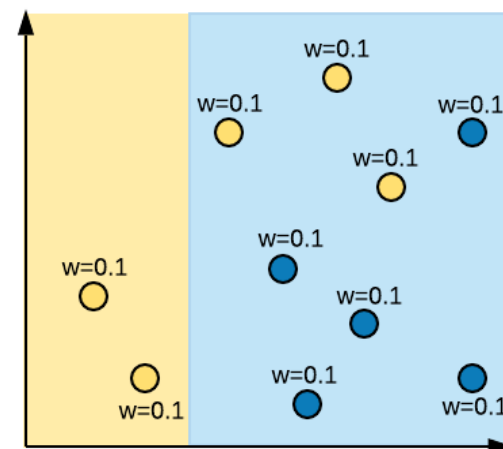
$$\alpha_m = \frac{1}{2} \ln \frac{1 - \text{err}_m}{\text{err}_m}$$

例如, 本例中, $\text{err}_1 = 0.3$, 则 $\alpha_1 = 0.42$ 。

For example, in this case, we have $\text{err}_1 = 0.3$, so $\alpha_1 = 0.42$.

- 一般说来, α 越大, 误差越小, 等价地, 置信度越高。当误差 $\text{err} = 0.5$ (随机猜测) 时, $\alpha = 0$ 。

Generally, higher α indicates lower error, or equivalently, more trust. When $\text{err} = 0.5$ (random guess), $\alpha = 0$.



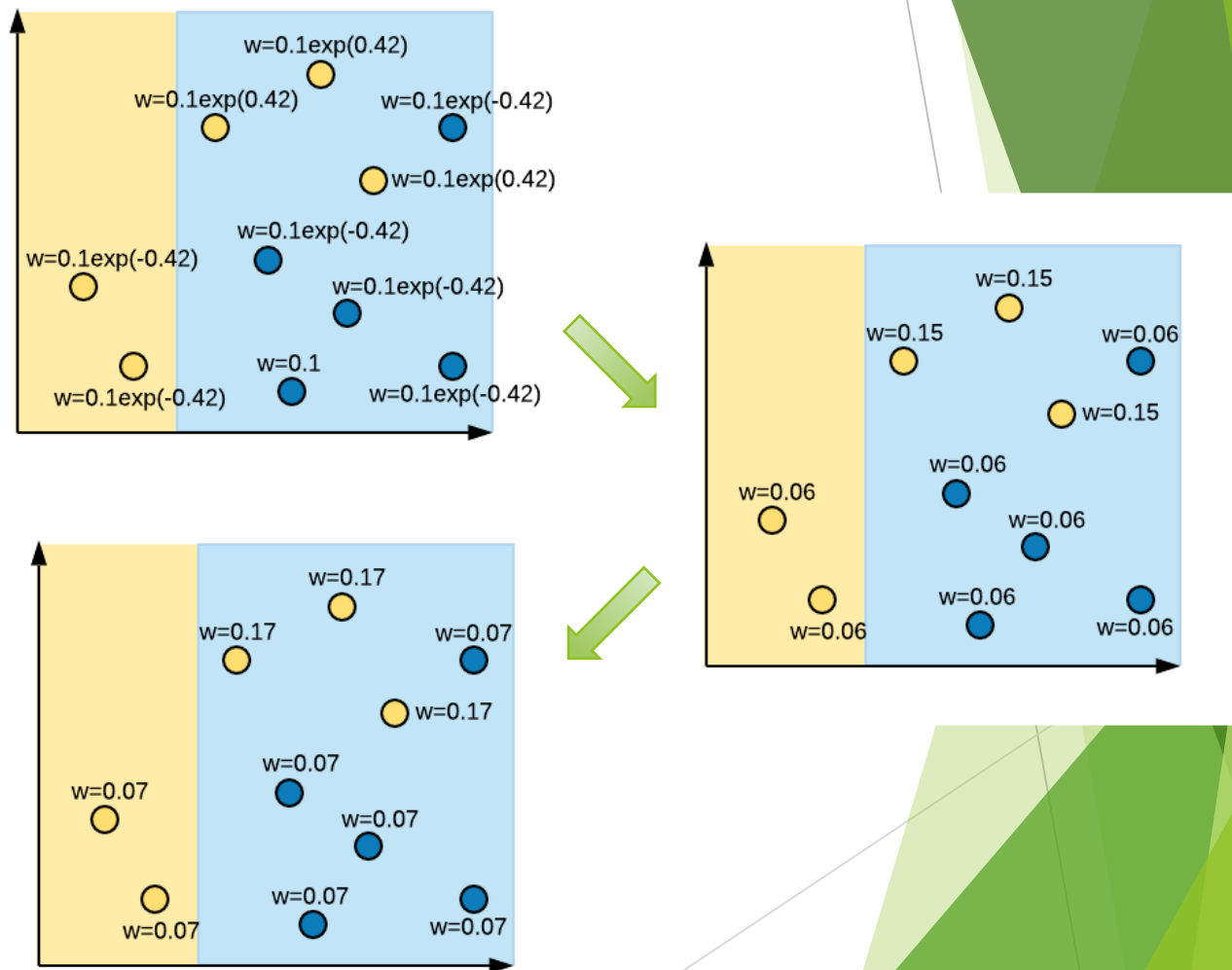
AdaBoost

► 步骤5: 调整样本权重:

- 对于预测正确的样本, 权重 $w_i^{(m+1)} = w_i^{(m)} \cdot e^{-\alpha_m}$;
- 对于预测错误的样本, 权重 $w_i^{(m+1)} = w_i^{(m)} \cdot e^{\alpha_m}$;
- 归一化所有权重 w_i , 使其和为1。

Step 5: Adjust the weights:

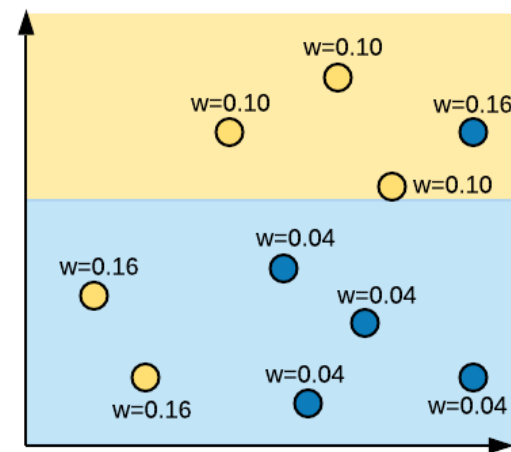
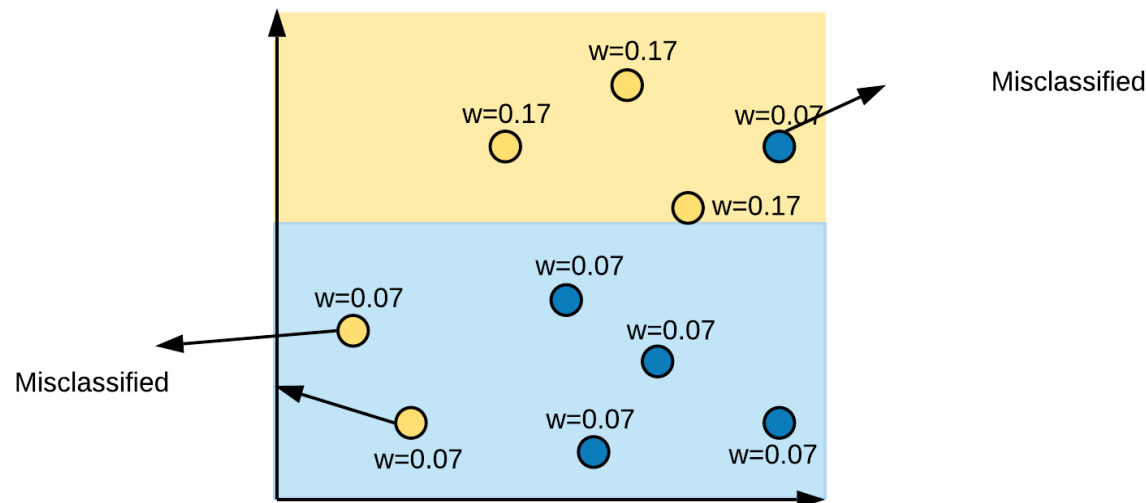
- For samples which were predicted correctly: $w_i^{(m+1)} = w_i^{(m)} \cdot e^{-\alpha_m}$;
- For samples which were predicted incorrectly: $w_i^{(m+1)} = w_i^{(m)} \cdot e^{\alpha_m}$;
- Normalize w_i 's to sum to 1.



AdaBoost

- 假设对第二个弱分类器而言，其分类情况如右图所示，重复上面步骤，可计算 $\text{err}_2 = 0.21$ ， $\alpha_2 = 0.42$ ，相应样本的权重也如右图所示。

Assume that for the second weak learner, its classification result is shown in the right figure. Repeating the steps above, we can calculate that $\text{err}_2 = 0.21$, $\alpha_2 = 0.42$, and the corresponding sample weights are also shown in the right figure.



AdaBoost

- 直观的讲，对于AdaBoost而言，每次迭代后，被错误分类的样本权重会提高，从而增加其重要性；被正确分类的样本权重会降低，从而减少其重要性。

Intuitively speaking, for AdaBoost, after each iteration, importance of wrongly classified samples is increased by increasing their weights and importance of correctly classified samples is decreased by decreasing their weights.

- 迭代结束后，对每个样本 x ，计算每个分类器的预测结果 $h_m(x)$ ，最终预测由加权预测和的符号决定，公式如下：

After the iteration, for each sample x , compute the prediction of each classifier $h_m(x)$. Final prediction is the sign of the sum of weighted predictions, given as:

$$\text{sign}(\alpha_1 \cdot h_1(x) + \alpha_2 \cdot h_2(x) + \cdots + \alpha_M \cdot h_M(x))$$

AdaBoost

- 我们再来总结一下AdaBoost算法，如右图所示：

Now we summarize the AdaBoost algorithm as in the right figure:

Consider we have a dataset of N samples.

Sample i has weight w_i . There are M classifiers in the ensemble.

1. Initialize weights of data samples: $w_i = \frac{1}{N}$
2. For $m = 1, \dots, M$:
 - 1) Learn classifier using current weights w_i 's
 - 2) Compute the weighted error: $\text{err}_m = \frac{\sum_i w_i(\text{incorrect})}{\sum_i w_i}$
 - 3) Compute $\alpha_m = \frac{1}{2} \log_e \left(\frac{1-\text{err}_m}{\text{err}_m} \right)$
 - 4) For samples which were predicted correctly: $w_i = w_i e^{-\alpha_m}$
 - 5) For samples which were predicted incorrectly: $w_i = w_i e^{\alpha_m}$